

MAML-Select: An Online Adaptive Client Selection Method for Federated Learning via Meta-Learning

Abstract—Federated Learning (FL) enables collaborative model training across distributed edge data, but client selection remains difficult when devices differ in data distribution, compute capacity, energy state, and communication latency. Selecting only the fastest or most statistically useful clients can slow convergence, increase resource cost, or reduce participation coverage. System-aware selectors such as FedCS and TiFL address stragglers, while utility-aware methods such as Oort, FedCor, CriticalFL, and FedGCS use loss, correlation, critical-period, or generative signals to improve training. However, many existing methods rely on fixed scoring rules, costly modelling stages, or feedback that does not explicitly adapt the selector before every round. The remaining gap is a lightweight policy that can rapidly re-rank clients as their utility and system state change during FL. We propose MAML-Select, an online adaptive selector that treats each FL round as a meta-learning task. A compact 6-64-64-1 MLP is adapted with a first-order MAML-style support update from recent local-loss and latency feedback, then used to score available clients before a query-set meta-update. Across Fashion-MNIST, CIFAR-10, and CIFAR-100 with 100 clients, $K = 10$, Dirichlet $\alpha = 0.5$, and three seeds, MAML-Select reduces cumulative TFLOPs by up to 21.5% and modelled energy/carbon by up to 24.7% relative to FedAvg, with reductions observed on all three datasets. Accuracy remains competitive with FedAvg and FedGCS, with the largest trade-off on CIFAR-10. These findings suggest that meta-adaptive client selection can make edge AI deployments more resource-conscious while retaining broad client coverage.

Impact Statement—Federated learning is central to privacy-preserving AI in mobile health, intelligent transportation, industrial sensing, and smart-city systems, but these deployments often face tight compute, energy, and latency budgets. MAML-Select contributes an adaptive selection mechanism that reduces unnecessary client computation while preserving broad participation coverage. By using recent training feedback to re-rank clients, the method supports FL systems that respond to changing device conditions instead of relying on a fixed rule. The reported energy and carbon values follow the declared simulator, device-tier, and grid-intensity setup, giving a transparent basis for comparing selection policies. The broader impact is a more resource-aware path for deploying distributed AI at the edge, especially when model updates must remain efficient, reproducible, and privacy-preserving.

Index Terms—Federated learning, client selection, meta-learning, and computational efficiency.

I. INTRODUCTION

FEDERATED Learning (FL) has emerged as an important framework for privacy-preserving distributed intelligence [1]. Clients train models locally and share only model updates. However, they differ in hardware, network conditions, and local data. This creates statistical drift under non-IID data and straggler delays under synchronous training [2]–[6]. Despite these advances, the problem of *whom to select* each round remains open.

Client selection chooses a subset $\mathcal{S}_t \subset \{1, \dots, N\}$ each round. Client utility is non-stationary: a useful client can become redundant, while an overlooked client can later become important. System-aware methods improve feasible participation but may bias selection away from slow yet data-rich clients [7], [8]. Utility-aware methods prioritize information gain, but may incur high latency or expensive server-side modelling [9]–[13]. Learning-based selectors use histories, features, or reinforcement signals [14]–[16]. An efficient round-by-round mechanism for adapting the ranking policy to evolving utility remains underexplored.

We propose MAML-Select, a client-selection framework based on a MAML-style fast adaptation step [17]. It learns an initialization for a policy network and adapts the policy from recent feedback before the next ranking decision. Unlike RL-based or contextual-bandit selectors, MAML-Select uses rapid gradient-based policy adaptation. Unlike FL personalization [18], it adapts the selector, not the client model. This is useful when devices have limited compute, strict deadlines, or intermittent connectivity. The major contributions can be summarized as follows.

- 1) We formulate the FL client selection problem as a meta-learning task and introduce the **MAML-Select** algorithm with rapid gradient-based adaptation for online client selection.
- 2) We demonstrate that MAML-Select *lowers cumulative training TFLOPs* by up to 21.5% and modelled energy/carbon by up to 24.7% relative to FedAvg, with paired-test support for the resource reductions.
- 3) Experiments on Fashion-MNIST, CIFAR-10, and CIFAR-100 under non-IID settings ($\alpha = 0.5$, three seeds) show that MAML-Select delivers accuracy competitive with FedAvg and FedGCS at lower computational cost, and higher accuracy than FedCS, Oort, TiFL, and FedCor in the reported runs; λ -sensitivity and feature-ablation studies support the design choices.

Organization: The rest of this paper is organized as follows. Section II reviews related work on client selection in FL. Section III details the system model and problem formulation. The proposed MAML-Select algorithm is described in Section IV, followed by evaluation and results in Section V. Section VI concludes this paper and discusses future research directions.

II. RELATED WORK

1) System-Aware Selection: System-aware selection addresses stragglers. FedCS selects feasible clients under resource constraints, while TiFL groups devices into online performance tiers [7], [8]. Recent studies show similar concerns in resource-constrained medical IoT and asynchronous FL with

heterogeneous objectives [4], [5]. MAML-Select uses such system signals while learning how much they should matter in the current training phase.

2) *Data-Aware and Utility-Based Selection*: Data-aware methods select clients for model utility rather than only speed. Oort prioritizes useful and fast clients, and FedCor models correlations between client losses [9], [10]. CriticalFL uses critical learning periods, while FedGCS searches for client combinations in a learned continuous space [12], [13]. Other work couples selection with compression or aggregation design [11], [19]. MAML-Select instead targets rapid adaptation of the ranking policy as client utility evolves.

3) *Learning-Based and Meta-Learning Approaches*: Learning-based selectors improve from observed feedback. FAVOR uses deep Q-learning, while neural contextual bandits learn from client features and rewards [14], [15]. Recent work also studies deep-RL selection for robustness and fairness-aware inclusive participation [16], [20]. Personalized FL is related but adapts models to clients; MAML-Select adapts the server-side ranking policy [18].

Table I consolidates these distinctions. For methods without a stated closed-form selection complexity, it reports the dominant server-side operation.

Difference from prior work: MAML-Select meta-learns the selection policy h_ϕ . Unlike heuristic, RL, contextual-bandit, and generative selectors, it uses a MAML inner update to convert recent selected-client feedback into fast policy weights before the next cohort is scored. A fixed heuristic or bandit can update rewards over time, but it does not explicitly adapt the scoring-network parameters from a support set immediately before selection. Its contribution is rapid policy adaptation under evolving client utility, not personalized model training.

III. SYSTEM MODEL AND PROBLEM FORMULATION

We consider a central server and N heterogeneous clients indexed by $\mathcal{N} = \{1, 2, \dots, N\}$. Client i stores a local dataset $\mathcal{D}_i$ with n_i samples. The global objective is to minimize the empirical risk over the distributed data as

$$\min_{\theta \in \mathbb{R}^d} F(\theta) = \sum_{i=1}^N \frac{n_i}{n} F_i(\theta), \quad (1)$$

where $\theta \in \mathbb{R}^d$ denotes the global model, $n = \sum_{i=1}^N n_i$, and $F_i(\theta) = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell(\theta; x_{i,j}, y_{i,j})$ is the local empirical risk.

At round t , let $\mathcal{A}_t \subseteq \mathcal{N}$ denote the available clients and $\mathcal{S}_t \subseteq \mathcal{A}_t$ the selected cohort. The server maintains the global model and the selection policy, while clients keep their data locally and return only model updates, local loss summaries, and resource-state feedback. Selection is therefore a server-side decision made before each broadcast, under the usual cross-device FL constraint that only a small fraction of available clients can train in a round.

Client i has latency $T_{i,t} = T_{i,t}^{comp} + T_{i,t}^{comm}$, where $T_{i,t}^{comp} = \frac{En_i C_{model}}{f_{i,t}}$ and $T_{i,t}^{comm} = \frac{M}{B_{i,t}} + \delta_{i,t}$. Here, E is the number of local epochs, C_{model} is the number of FLOPs per sample, $f_{i,t}$ is the compute rate, M is the update size, $B_{i,t}$ is the uplink

bandwidth, and $\delta_{i,t}$ is the propagation delay. In synchronous FL, the slowest selected client determines the round latency:

$$T_t^{round} = \max_{i \in \mathcal{S}_t} T_{i,t}, \quad (2)$$

We report simulated latency and TFLOPs as computation indicators. Energy and carbon are derived from the device-tier power and grid-intensity assumptions stated in Section V.

Let $a_{i,t}$ indicate whether client i is selected. Under policy π_ϕ , the constrained client-selection problem is

$$\begin{aligned} \min_{\pi_\phi} \quad & \sum_{t=1}^T [F(\theta_{t+1}) + \lambda T_t^{round}] \\ \text{s.t.} \quad & \sum_{i=1}^N a_{i,t} = K, \quad a_{i,t} \in \{0, 1\}, \\ & a_{i,t} \leq \mathbf{1}\{i \in \mathcal{A}_t\}, \quad \mathcal{S}_t = \{i : a_{i,t} = 1\}. \end{aligned} \quad (3)$$

Here, K is the cohort size, T is the number of rounds, and $\lambda \geq 0$ controls the latency-loss trade-off. A larger λ places more emphasis on latency, while $\lambda = 0$ yields loss-only selection. We assume $|\mathcal{A}_t| \geq K$. The model θ_{t+1} results from local training and aggregation over $\mathcal{S}_t$. The problem remains challenging because the action space is combinatorial and client utility evolves during training.

IV. METHODOLOGY: MAML-SELECT ALGORITHM

Building on (3), MAML-Select treats client selection as a meta-learning problem. Client utility changes as training progresses. However, the mechanism for selecting useful clients from recent feedback can be learned.

1) *State Space*: The server constructs a state vector $\mathbf{s}_{i,t} \in \mathbb{R}^6$ for each client i at round t . It contains local training loss $\hat{L}_{i,t}$, gradient norm $\|\nabla F_i\|_2$, estimated latency $\hat{T}_{i,t}$, battery level $b_{i,t}$, selection frequency $q_{i,t}$, and staleness $\tau_{i,t}$ (rounds since last selected) [11], [15]. Loss and gradient norm characterize statistical utility. Latency and battery level characterize system constraints. Selection frequency and staleness expose participation imbalance. Each FL round t is treated as a task $\mathcal{T}_t$. The goal is to rank clients so that the top- K cohort minimizes the per-round cost.

MAML-Select trains a meta-policy network $h_\phi(\mathbf{s}_{i,t})$, parameterized by ϕ , that learns an initialization adaptable to the current round using data from the preceding round. The policy is a 6-64-64-1 multilayer perceptron with ReLU activations and 4,673 trainable parameters. The support set represents the past experience: $\mathcal{D}_{sup}(t) = \{(\mathbf{s}_{i,t-1}, c_{i,t-1}) \mid i \in \mathcal{S}_{t-1}\}$, containing client states and observed costs from round $t-1$. The candidate set represents the current decision: $\mathcal{D}_{cand}(t) = \{\mathbf{s}_{i,t} \mid i \in \mathcal{A}_t\}$, containing the states of available clients.

2) *Cost Function*: The per-client cost $c_{i,t}$ supplies feedback to the ranking policy. Consistent with (3), it is defined as

$$c_{i,t} = \underbrace{\lambda \max(0, T_{i,t} - T_{target})}_{\text{latency penalty}} - \underbrace{\Delta_{i,t}}_{\text{local loss reduction}}. \quad (4)$$

Here, $\Delta_{i,t} = \hat{L}_{i,t}^{before} - \hat{L}_{i,t}^{after}$ is the observed local loss reduction for selected client i . It is a client-level proxy for statistical utility and does not require a server-held validation set. The target T_{target} is a soft deadline. Minimizing $c_{i,t}$ favors lower latency and larger local loss reduction.

TABLE I
COMPARISON OF REPRESENTATIVE METHODS AND MAML-SELECT.

Method	Primary objective	Adaptation mechanism	Selection complexity or dominant operation	Optimization strategy
FedCS [7]	Increase feasible updates under resource constraints	Resource estimates before selection	Resource-constrained subset search	Deadline-aware scheduling
TiFL [8]	Mitigate stragglers while preserving accuracy	Online tier updates	Tiering and probabilistic sampling	Adaptive tier-based selection
Oort [9]	Improve time-to-accuracy	Utility and speed feedback	Utility scoring and guided sampling	Exploration and exploitation
FedCor [10]	Accelerate convergence under non-IID data	Loss-correlation model	Gaussian-Process covariance operations	GP-based active selection
FAVOR [14]	Counterbalance non-IID bias	Experience-driven policy updates	DQN inference and training	Deep Q-learning
NCCB [15]	Learn contextual client combinations	Client features and reward feedback	LSH feature extraction and bandit updates	Neural contextual combinatorial bandit
FedCG [11]	Couple selection and gradient compression	Capability-aware compression updates	Submodular optimization and linear programming	Joint iterative optimization
CriticalFL [12]	Exploit critical learning periods	Period-aware participation	Base-selector dependent	Selector augmentation
FedGCS [13]	Optimize client combinations	Search in a learned continuous space	Latent-space optimization and beam search	Gradient-based generative selection
Kazemi et al. [16]	Improve robustness under poisoning attacks	Experience-driven policy updates	Deep-RL inference and training	Deep reinforcement learning
Vox Populi [20]	Promote fair and inclusive participation	Fairness-aware client selection	Framework-dependent	Inclusive selection framework
MAML-Select	Adapt rankings under evolving client utility	Fast adaptation from recent round feedback	$\mathcal{O}(N \phi)$ scoring plus a gradient-adaptation step	Gradient-based meta-learning

A. The MAML-Select Algorithm

The algorithm proceeds in three phases per round:

Phase 1 (Inner Loop Adaptation (Look-Ahead)): At the start of round t , the server adapts the meta-policy ϕ_t on $\mathcal{D}_{sup}(t)$ using one gradient-descent step:

$$\phi'_t = \phi_t - \beta \nabla_{\phi_t} \mathcal{L}_{sup}(h_{\phi_t}; \mathcal{D}_{sup}(t)), \quad (5)$$

where $\mathcal{L}_{sup}$ is the MSE loss between predicted scores and observed costs from round $t-1$, and β is the inner learning rate.

Statement 1. Let $g_t(\phi) = \mathcal{L}_{sup}(h_{\phi}; \mathcal{D}_{sup}(t))$. If g_t is L -smooth and $0 < \beta < 2/L$, the inner update in (5) satisfies

$$g_t(\phi'_t) \leq g_t(\phi_t) - \beta \left(1 - \frac{L\beta}{2}\right) \|\nabla g_t(\phi_t)\|_2^2.$$

This is a selector-level descent property. It shows that one support-set step can reduce the round-specific policy loss before ranking current candidates. It motivates rapid adaptation. It is not a global FL convergence guarantee for the client model.

Phase 2 (Client Selection): The adapted policy $h_{\phi'_t}$ scores the available clients and selects the K clients with the lowest predicted costs:

$$\mathcal{S}_t = \text{Top-}K_{i \in \mathcal{A}_t} (-h_{\phi'_t}(\mathbf{s}_{i,t})). \quad (6)$$

Phase 3 (FL Execution and Outer Loop Update): The selected clients perform standard FL training. Their observed costs form $\mathcal{D}_{query}(t) = \{(\mathbf{s}_{i,t}, c_{i,t}) \mid i \in \mathcal{S}_t\}$. The query loss is also mean squared error. The meta-policy is updated on this query set:

$$\phi_{t+1} \leftarrow \phi_t - \eta \nabla_{\phi'_t} \mathcal{L}_{query}(h_{\phi'_t}; \mathcal{D}_{query}(t)), \quad (7)$$

where η is the meta-learning rate. We use a first-order MAML-style approximation. It applies the query gradient at ϕ'_t and omits second-order derivatives. The full procedure is detailed in Algorithm 1 and Fig. 1.

MAML-Select differs from fixed-rule selectors because its ranking policy is adapted before each decision. It also differs

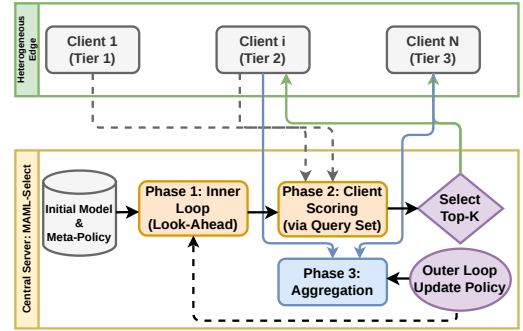


Fig. 1. System model and MAML-Select workflow. A central server observes client states, selects K clients from the available pool, aggregates their local updates, and uses returned local-loss and system feedback for support-set adaptation, candidate scoring, and query-set meta-update.

Algorithm 1 MAML-Select Algorithm

```

1: Initialize: Global model  $\theta_0$ ; meta-policy parameters  $\phi$ ; inner LR  $\beta$ ; outer LR  $\eta$ ; trade-off  $\lambda$ .
2: for round  $t = 1, \dots, T$  do
3:   Server broadcasts global model  $\theta_t$  to all clients.
4:   Collect state vectors  $\mathbf{s}_{i,t}$  from available clients.
5:   if  $t > 1$  then
6:     // Phase 1: Inner Loop Adaptation
7:     Construct  $\mathcal{D}_{sup}$  from round  $t-1$  results.
8:     Compute fast weights  $\phi'_t$  using Eq. 5.
9:   else
10:     $\phi'_t \leftarrow \phi_t$  // No adaptation for the first round
11:   end if
12:   // Phase 2: Client Selection
13:   Score clients:  $\text{score}_i = h_{\phi'_t}(\mathbf{s}_{i,t})$  for all  $i \in \mathcal{A}_t$ .
14:   Select lowest-cost clients:  $\mathcal{S}_t \leftarrow \text{Top-}K(-\text{scores})$ .
15:   // FL Training Round
16:   Receive local updates  $\{\Delta\theta_{i,t}\}_{i \in \mathcal{S}_t}$ , local losses, and system metrics.
17:   Aggregate:  $\theta_{t+1} \leftarrow \theta_t + \sum_{i \in \mathcal{S}_t} \frac{n_i}{\sum_{j \in \mathcal{S}_t} n_j} \Delta\theta_{i,t}$ .
18:   // Phase 3: Outer Loop Meta-Update
19:   Compute costs  $c_{i,t}$  using Eq. 4 for  $i \in \mathcal{S}_t$ .
20:   Construct  $\mathcal{D}_{query}(t) = \{(\mathbf{s}_{i,t}, c_{i,t}) \mid i \in \mathcal{S}_t\}$ .
21:   Meta-update:  $\phi_{t+1} \leftarrow \phi_t - \eta \nabla_{\phi'_t} \mathcal{L}_{query}(h_{\phi'_t}; \mathcal{D}_{query}(t))$ .
22: end for

```

from RL selectors because the update is an explicit gradient step on recent support feedback. The method is designed for rapidly changing client utility.

1) *Computational Complexity:* Let $P = |\phi|$ denote the number of meta-policy parameters. For the 6-64-64-1 MLP

used here, $P = 4,673$. A forward pass for one client costs $\mathcal{O}(P)$, so scoring N available clients costs $\mathcal{O}(NP)$. Selecting the best K clients can be implemented with a size- K heap in $\mathcal{O}(N \log K)$, or by partial sorting. The inner support update and outer query update each use at most the selected cohort and therefore cost $\mathcal{O}(KP)$. Thus the per-round server-side selector cost is

$$\mathcal{O}(NP + N \log K + 2KP). \quad (8)$$

Since P and K are fixed in the experimental protocol ($P = 4,673$, $K = 10$), MAML-Select scales linearly with the client pool size N . The memory cost is $\mathcal{O}(P + N + K)$ for the policy parameters, client scores, and selected cohort. This contrasts with correlation-matrix or Gaussian-Process selection stages whose covariance operations can grow superlinearly, and in common FedCor-style implementations cubically, with N .

V. EVALUATION AND RESULTS

We evaluate on three benchmarks: Fashion-MNIST using LightCNN-9 [21] (lightweight IoT task), CIFAR-10 using ResNet18 [22] (complex vision task), and CIFAR-100 using ResNet18 (100-class fine-grained task). $N = 100$ clients are simulated with non-IID data partition (Dirichlet $\alpha = 0.5$), selecting $K = 10$ clients per round for $T = 200$ rounds (150 rounds for CIFAR-100) with $E = 5$ local epochs and batch size 32. Table II reports mean $\pm$ standard deviation over repeated runs. The tests use seeds 42, 123, and 2026. Local models use PyTorch default initialization and SGD with learning rate 0.01, momentum 0.9, weight decay 10^{-4} , and a constant scheduler. The meta-policy uses one inner step with $\beta = 0.01$ and Adam for the outer update with $\eta = 0.001$. The selector seed is fixed to 2026. Compute rates follow three device tiers inspired by mobile benchmarking studies [23]: Tier 1 (20%, $1.0\times$), Tier 2 (50%, $2.0\times$), and Tier 3 (30%, $4.0\times$). Resource accounting uses the same tiers: Tier 1 devices use 4 W power and 18 Wh batteries, Tier 2 uses 7 W and 30 Wh, and Tier 3 uses 12 W and 48 Wh. Round time is straggler-bound by the slowest selected client. Round energy sums the selected clients' modelled local-training energy, and carbon uses 475 gCO₂e/kWh. The target latency deadline T_{target} is set to the average round latency of Tier 2 clients, and the trade-off parameter $\lambda = 0.5$. Baselines include FedAvg (random selection), FedCS (system-aware deadline-constrained), Oort (utility-based with importance sampling), TiFL (tier-based), FedCor (correlation-based via Gaussian Processes), FedGCS (generative selection), and CriticalFL (critical-period cohort scaling).

A. Accuracy and Resource Cost

We estimate the client-training computation per round as

$$\text{TFLOPs}_{total} = \sum_{t=1}^T \sum_{i \in S_t} E \cdot n_i \cdot C_{model} \times 10^{-12}, \quad (9)$$

where C_{model} is the per-sample cost. For ResNet18 on CIFAR-10, one selected-client round is approximately 4.13 TFLOPs. Modelled energy scales local-training time by per-tier power, and carbon scales energy by 475 gCO₂e/kWh.

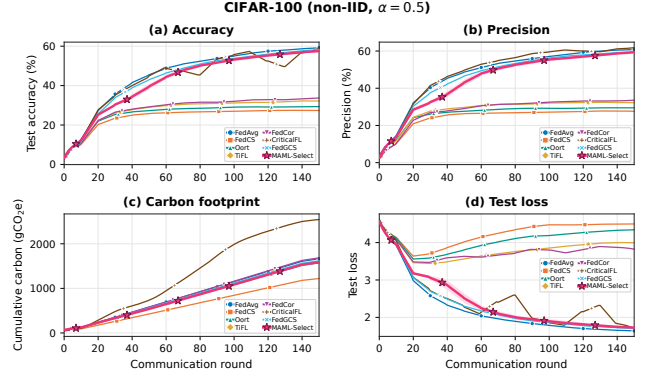


Fig. 2. CIFAR-100 convergence over 150 rounds. The panels report accuracy, precision, cumulative modelled carbon, and test loss.

Carbon-accounting metadata and emission units follow CodeCarbon v2.4.1 [24].

Table II summarises all methods. MAML-Select attains accuracy on par with FedAvg and FedGCS on Fashion-MNIST (90.1% vs. 90.2% and 90.7%) and CIFAR-100 (58.2% vs. 59.7% and 58.7%). It also exceeds FedCS, Oort, TiFL, FedCor, and CriticalFL on CIFAR-100 accuracy. On CIFAR-10, it accepts a moderate accuracy reduction (75.6% vs. FedAvg 79.4%) for substantially lower computational cost. This is the hardest trade-off case: ResNet18 training is expensive, and $\lambda = 0.5$ can favour lower-cost clients even when some costly clients contain useful class variation. Overall, MAML-Select offers a resource-aware operating point, staying close to the strongest accuracy baselines while reducing TFLOPs, modelled energy, and carbon across all reported non-IID settings. The meta-update overhead ($\epsilon \approx 0.001$ GFLOPs per round) is negligible relative to client training. Paired tests against FedAvg support the resource reductions on all datasets: TFLOP p -values are 0.009/0.002/0.012, and energy/carbon p -values are 0.007/0.001/0.003 for Fashion-MNIST/CIFAR-10/CIFAR-100.

Fig. 2 summarises the CIFAR-100 round-wise behaviour. MAML-Select remains close to FedAvg and FedGCS on accuracy, precision, and loss while staying in the efficient carbon cluster. The round-wise curves show that higher-cost selection does not automatically translate into better accuracy under this non-IID setting.

Fig. 3 shows the efficiency-accuracy trade-off relative to FedAvg. Points to the left of $x = 1$ use less cumulative training compute; points above $y = 0$ improve accuracy. MAML-Select is consistently in the lower-compute region. Relative to FedAvg over shared seeds, it lowers cumulative TFLOPs by 12.8%/21.5%/1.7% and modelled energy and carbon by 16.4%/24.7%/4.7% on Fashion-MNIST/CIFAR-10/CIFAR-100. The corresponding accuracy changes are $-0.1/-3.8/-1.5$ percentage points. Thus the main empirical gain is resource efficiency with competitive accuracy. This is the desired operating region for resource-constrained FL: the selector keeps broad data coverage while operating at a lower compute point than several utility-driven alternatives.

MAML-Select reaches 100% client coverage on every dataset. While its Jain index (0.61–0.78) is lower than those of FedAvg and FedGCS, a perfect coverage demonstrates that

TABLE II
BENCHMARK SUMMARY ACROSS NON-IID DATASETS. ACCURACY, PRECISION, RECALL, F1, AND COVERAGE ARE IN PERCENT; ENERGY IS IN WATT-HOUR (WH) AND CARBON IS IN GRAMS OF CARBON DIOXIDE EQUIVALENT (GCO₂E).

Method	Acc.	Prec.	Rec.	F1	TFLOPs	Energy	Carbon	Jain	Cov.
<i>Fashion-MNIST</i>									
FedAvg	90.22±0.58	90.55±0.20	90.22±0.58	90.22±0.51	140±1	5752±72	2732±34	0.95±0.00	100±0
FedCS	84.11±1.82	84.84±0.77	84.11±1.82	83.91±1.98	76±7	2772±211	1317±100	0.10±0.00	10±0
Oort	86.70±0.19	86.66±0.21	86.70±0.19	86.42±0.26	149±8	5963±456	2832±217	0.10±0.00	10±0
TiFL	86.73±0.34	86.62±0.26	86.73±0.34	86.51±0.34	147±6	6062±348	2879±165	0.11±0.00	12±0
FedCor	89.35±0.50	89.31±0.51	89.35±0.50	89.22±0.57	121±5	4845±204	2301±97	0.26±0.04	49±8
CriticalFL	90.16±0.47	90.24±0.38	90.17±0.47	90.11±0.42	304±143	12552±5860	5962±2784	0.98±0.01	100±0
FedGCS	90.65±0.29	90.73±0.16	90.65±0.29	90.63±0.26	136±4	5595±113	2658±54	0.95±0.02	100±0
MAML-Select	90.11±0.47	90.43±0.04	90.11±0.47	90.04±0.30	122±2	4809±79	2284±37	0.77±0.06	100±0
<i>CIFAR-10</i>									
FedAvg	79.43±1.84	80.01±1.28	79.43±1.84	79.36±1.78	8341±52	4798±24	2279±11	0.95±0.00	100±0
FedCS	50.45±2.57	50.88±2.60	50.45±2.57	49.87±2.33	4081±600	2073±262	985±125	0.10±0.00	10±0
Oort	60.19±5.18	60.84±4.97	60.19±5.18	59.69±4.61	8303±1923	4677±856	2222±406	0.10±0.00	10±0
TiFL	61.93±5.04	62.48±4.78	61.93±5.04	61.36±4.60	8200±2143	4800±1203	2280±572	0.11±0.00	12±0
FedCor	67.20±5.33	67.83±4.74	67.20±5.33	66.81±5.37	7748±695	4452±435	2115±206	0.20±0.02	31±5
CriticalFL	77.34±5.21	79.98±2.80	77.34±5.21	77.46±5.21	14239±4864	8205±2807	3897±1333	0.98±0.01	100±0
FedGCS	77.88±1.75	78.93±1.50	77.88±1.75	77.69±1.54	7656±278	4428±154	2103±73	0.90±0.03	100±0
MAML-Select	75.63±1.43	76.51±1.55	75.63±1.43	75.25±1.34	6549±118	3610±60	1715±29	0.61±0.01	100±0
<i>CIFAR-100</i>									
FedAvg	59.66±0.29	60.90±0.48	59.66±0.29	59.13±0.40	6331±6	3626±18	1722±9	0.94±0.00	100±0
FedCS	27.52±0.29	27.93±0.10	27.52±0.29	26.39±0.09	5334±23	2659±12	1263±6	0.10±0.00	10±0
Oort	29.41±1.73	29.82±1.77	29.41±1.73	28.13±1.73	5955±189	3421±181	1625±86	0.10±0.00	10±0
TiFL	32.12±0.30	33.07±0.50	32.12±0.33	31.02±0.42	5986±28	3499±14	1662±5	0.11±0.00	12±0
FedCor	34.18±0.24	33.89±0.26	34.18±0.60	32.93±0.20	6428±32	3668±18	1742±3	0.12±0.00	14±0
CriticalFL	48.84±1.58	60.56±1.89	48.84±1.58	48.07±2.32	9469±392	5438±214	2583±102	0.97±0.00	100±0
FedGCS	58.66±0.11	60.24±0.03	58.66±0.11	58.08±0.20	6140±24	3526±4	1675±2	0.82±0.02	100±0
MAML-Select	58.15±0.51	59.87±0.55	58.15±0.51	57.66±0.33	6224±20	3457±6	1642±3	0.78±0.03	100±0

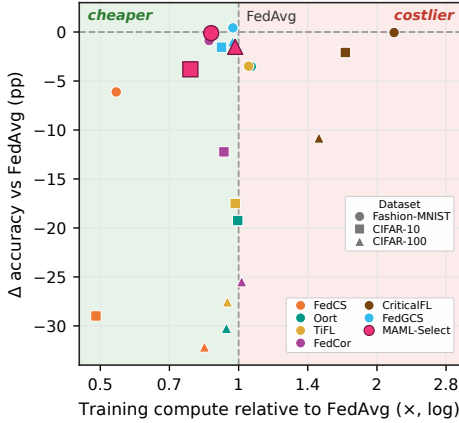


Fig. 3. Efficiency-accuracy trade-off relative to FedAvg. $x < 1$ indicates lower cumulative training compute, while y reports accuracy change in percentage points.

the policy does not permanently exclude any clients.

B. Selector Scaling

To validate the complexity expression in (8), we measured the server-side selection overhead of MAML-Select for 50 rounds. We varied the client pool size as $N \in \{20, 40, 80, 100\}$ and fixed the sampling ratio at $K/N = 0.1$. Fig. 4 shows that the measured overhead stays between approximately 11 and 13 ms per round. Thus, the meta-policy adds only a small server-side cost. The analytical cost is $NP + N \log K + 2KP$, where $P = 4,673$ is the number of policy parameters. In the main experiments, both P and $K = 10$ are fixed. The dominant term therefore scales linearly with N . The one-step inner loop is also deliberate: adding more inner steps would multiply the adaptation term and increase per-round server work. This

makes the adaptive scoring step practical for the evaluated FL setting. Its cost is small compared with local client training.

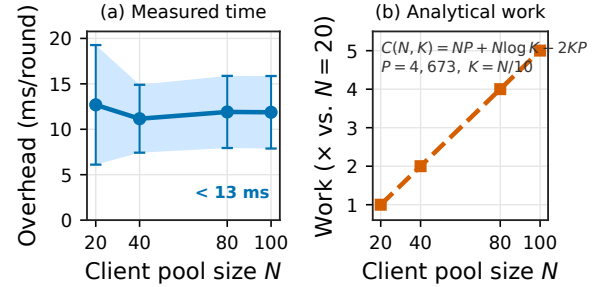


Fig. 4. MAML-Select selector scaling. Panel (a) reports measured overhead with 95% confidence intervals over 10 rounds. Panel (b) reports the normalized operation count from Eq. 8.

C. Sensitivity and Ablation

The sensitivity and ablation experiments test whether the proposed selector is stable when its objective or input representation is perturbed. The first study varies the cost trade-off λ . The second removes one state feature at a time. Together, they examine the two design choices most specific to MAML-Select: the resource-aware cost and the six-feature policy state.

Fig. 5 shows the role of λ as an accuracy-efficiency control in MAML-Select. All four curves are normalized to the $\lambda = 0.1$ setting. This allows accuracy, compute, energy, and fairness to be compared on the same scale. As λ increases, the selector gives more weight to lower-cost clients. Compute and energy therefore decrease steadily. Accuracy remains stable across the tested range, with a spread of about one percentage point. Fairness is more sensitive at larger λ values. This shows that λ can tune the operating point without destabilizing

learning. We use $\lambda = 0.5$ as the default setting. It keeps accuracy close to the best observed value. It also reduces computation relative to $\lambda = 0.1$. At the same time, it avoids the sharper fairness loss observed at $\lambda = 1$ and $\lambda = 5$.

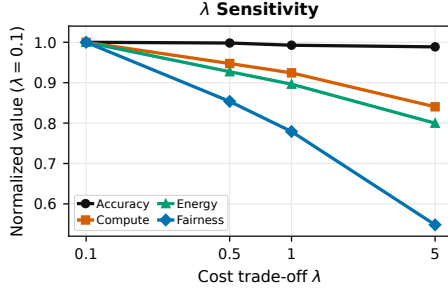


Fig. 5. λ sensitivity on Fashion-MNIST. Curves are normalized to the value at $\lambda = 0.1$ and report accuracy, compute, energy, and fairness.

Fig. 6 and Table III report the state-feature ablation. Each variant removes one feature from the MAML-Select state vector. Accuracy changes remain small, within about 0.35 percentage points. This indicates that the learned policy is not fragile to a single missing feature. The largest drops occur when battery or loss feedback is removed. Battery helps the policy avoid energy-poor clients, while loss feedback identifies statistically useful updates. Thus, the selector uses both system information and learning feedback. Removing frequency or staleness gives a small accuracy gain in some runs. However, these variants reduce fairness and coverage stability. We therefore keep all six features in the final state vector.

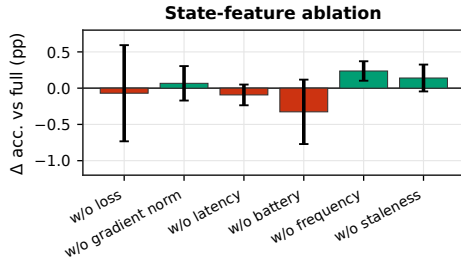


Fig. 6. State-feature ablation on Fashion-MNIST. Each bar reports the accuracy change relative to the full six-feature state vector.

TABLE III

STATE-FEATURE ABLATION ON FASHION-MNIST ($n = 3$). EACH ROW REMOVES ONE FEATURE FROM THE SIX-DIMENSIONAL STATE VECTOR; THE FULL VECTOR IS THE REFERENCE.

Variant	Acc. (%)	TFLOPs	Jain
Full state vector	90.23 ± 0.55	122	0.78
w/o loss	90.16 ± 0.66	123	0.79
w/o gradient norm	90.30 ± 0.24	122	0.78
w/o latency	90.14 ± 0.14	119	0.70
w/o battery	89.90 ± 0.44	122	0.79
w/o frequency	90.47 ± 0.13	115	0.65
w/o staleness	90.37 ± 0.19	123	0.78

VI. CONCLUSION AND FUTURE WORK

We introduced MAML-Select, an online adaptive client-selection framework for heterogeneous federated learning. The method learns a lightweight ranking policy that uses both

learning feedback and system-state information. Before each selection decision, the policy is adapted with recent support-set feedback and then used to score the available clients. This gives the server a practical way to balance accuracy, computation, energy, carbon cost, and participation coverage during training. Across Fashion-MNIST, CIFAR-10, and CIFAR-100, MAML-Select achieves competitive accuracy while reducing cumulative computation, modelled energy, and carbon compared with higher-cost selection strategies. The λ -sensitivity study shows that the cost trade-off can be tuned without destabilizing accuracy. The feature-ablation study further shows that both statistical and system features contribute to stable selection behaviour. These results support MAML-Select as a compact and resource-aware client-selection method for edge-oriented FL. Future work will extend the evaluation to sound datasets, larger client pools, and more diverse real-device conditions.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.
- [2] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, “SCAFFOLD: Stochastic controlled averaging for federated learning,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 5132–5143.
- [3] V. S. Mai, R. J. La, and T. Zhang, “A study of enhancing federated learning on non-IID data with server learning,” *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 11, pp. 5589–5604, 2024.
- [4] S. Tapwal, “Stragglers reimaged: Explainability-driven adaptive federated learning for resource constrained IoMT system,” *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1002–1011, 2026.
- [5] A. Forootani, R. Iervolino, and M. Tipaldi, “Asynchronous federated learning with nonconvex client objective functions and heterogeneous dataset,” *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 2610–2624, 2026.
- [6] P. Borazjani, M. S. Fouladgar, and R. Ghanbarzadeh, “Redefining non-IID data in federated learning for computer vision tasks,” *IEEE Transactions on Artificial Intelligence*, 2026, early Access.
- [7] T. Nishio and R. Yonetani, “Client selection for federated learning with heterogeneous resources in mobile edge,” in *IEEE International Conference on Communications (ICC)*. IEEE, 2019, pp. 1–7.
- [8] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, and Y. Cheng, “TiFL: A tier-based federated learning system,” in *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*, 2020, pp. 125–136.
- [9] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, “Oort: Efficient federated learning via guided participant selection,” in *15th USENIX Symposium on Operating Systems Design and Implementation*, 2021, pp. 19–35.
- [10] M. Tang, X. Ning, Y. Wang, J. Sun, Y. Wang, H. Li, and Y. Chen, “FedCor: Correlation-based active client selection strategy for heterogeneous federated learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 2022, pp. 10 102–10 111.
- [11] Y. Xu, Z. Jiang, H. Xu, Z. Wang, C. Qian, and C. Qiao, “Federated learning with client selection and gradient compression in heterogeneous edge systems,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5446–5461, 2024.
- [12] G. Yan, H. Wang, X. Yuan, and J. Li, “CriticalFL: A critical learning periods augmented client selection framework for efficient federated learning,” in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 5034–5044.
- [13] Z. Ning, C. Tian, M. Xiao, W. Fan, P. Wang, L. Li, P. Wang, and Y. Zhou, “FedGCS: A generative framework for efficient client selection in federated learning via gradient-based optimization,” in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, 2024, pp. 4760–4768.
- [14] H. Wang, Z. Kaplan, D. Niu, and B. Li, “Optimizing federated learning on non-IID data with reinforcement learning,” in *IEEE INFOCOM*. IEEE, 2020, pp. 1698–1707.

- [15] Q. Pan, H. Cao, Y. Zhu, J. Liu, and B. Li, "Contextual client selection for efficient federated learning over edge devices," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 6538–6548, 2024.
- [16] F. Kazemi, A. Badiei, and H. Kebriyai, "Robust peer-to-peer federated learning with deep reinforcement learning based client selection against data poisoning attacks," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 262–271, 2026.
- [17] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 1126–1135.
- [18] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," *Advances in Neural Information Processing Systems*, vol. 33, pp. 3557–3568, 2020.
- [19] N. T. Tran, L. H. T. Tam, and M. T. Thai, "FedImp: Enhancing federated learning convergence with impurity-based weighting aggregation," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1652–1665, 2026.
- [20] S. Arisdakessian, O. A. Wahab, A. Mourad, and H. Otrouk, "Towards vox populi in federated learning: A fair and inclusive client selection framework," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1997–2011, 2026.
- [21] X. Wu, R. He, Z. Sun, and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 11, pp. 2884–2896, 2018.
- [22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2016, pp. 770–778.
- [23] A. Ignatov, R. Timofte, A. Kulik, S. Yang, K. Wang, F. Baum, M. Wu, L. Xu, and L. Van Gool, "AI Benchmark: Running deep neural networks on android smartphones," in *European Conference on Computer Vision Workshops*, 2018, pp. 288–314.
- [24] B. Courty, V. Schmidt, S. Luccioni, Goyal-Kamal, MarionCoutarel, B. Feld, J. Lecourt, LiamConnell, A. Saboni, Inimaz, supatomic, M. Leval, L. Blanche, A. Cruveiller, ouminasara, F. Zhao, A. Joshi, A. Bogroff, H. de Lavoreille, N. Laskaris, E. Abati, D. Blank, Z. Wang, A. Catovic, M. Alencon, M. Stechly, C. Bauer, L. O. N. de Araujo, JPW, and MinervaBooks, "mlco2/codecarbon: v2.4.1," May 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.11171501>